

PLANEAMENTO E GESTÃO DE PROJETO · 2026/27

Guia de estudo: justificação das decisões

Gestão de coleções de álbuns de música em formato físico
Etapa 1 — Desenho inicial e pesquisa de tecnologias

Grupo 12 · Rodrigo, Reis, Miguel, Ze, Edson
Complementa o `Guia-de-Estudo-Etapa1.pdf`: aqui está o **porquê** de cada escolha.

Como usar este guia

Na apresentação, o professor pergunta sobretudo **porquê**. Este guia reúne todas as decisões da Etapa 1 em cartões com o mesmo formato:

- **Decisão:** o que se escolheu.
- **Alternativas:** o que se considerou e se pôs de lado.
- **Porquê:** a justificação, em poucas frases.
- **Base:** a parte do enunciado, a user story (US), o requisito não funcional (RNF) ou a restrição (RI) que sustenta a decisão.
- **Se perguntarem:** a pergunta provável e a resposta curta, a vermelho.

Regra para responder a qualquer pergunta: 1) diz a decisão numa frase; 2) diz o porquê, ligado a uma US, RNF ou frase do enunciado; 3) se fizer sentido, diz que alternativa foi posta de lado e porquê. Se não souberes, diz o que está assumido e que é uma suposição documentada.

Índice

1 Processo e organização (D1–D8)
2 Arquitetura e tecnologias (D9–D17)
3 Serviços externos (D18–D25)
4 Casos de uso (D26–D31)
5 Modelo de dados (D32–D49)
6 Restrições de integridade (D50–D52)
7 Fluxos de trabalho (D53–D60)
8 Esboços da interface (D61–D66)
9 Ferramentas de trabalho (D67–D74)
10 Suposições (S1–S6)
11 Resumo numa página

1 Processo e organização

D1 Modelo de processo: Scrum

Decisão	Desenvolver em PTIR com Scrum, em 3 sprints de cerca de um mês.
Alternativas	Cascata; incremental; espiral.
Porquê	Os quatro critérios de escolha apontam para Scrum: **cliente** participativo, com demonstrações no fim de cada sprint; **pessoas**: equipa de 5, dentro dos 3 a 9 recomendados; **produto**: requisitos que podem mudar; **ambiente**: semestre com data de fim fixa. A cascata exige requisitos estáveis e só entrega no fim; o incremental planeia os incrementos à partida; a espiral centra cada ciclo na análise de risco, pesada para 5 pessoas num semestre.
Base	Enunciado, secção 1 ("inspirado no modelo de processo Scrum, com três sprints"; "o cliente pode solicitar alterações"); Tabela 4 do relatório.
Se perguntarem	"Porque não cascata?" — Porque o cliente pode mudar requisitos e quer ver demonstrações a meio; a cascata só entrega no fim.

D2 O gestor de projeto faz parcialmente de Product Owner

Decisão	Um membro da equipa ordena as user stories e distribui-as pelos sprints.
Alternativas	Um Product Owner externo (o cliente).
Porquê	É o que o enunciado define; o cliente participa nas demonstrações, mas a priorização fica na equipa.
Base	Enunciado, secção 4 e nota 20.

D3 Divisão do trabalho por artefacto

Decisão	Cada pessoa é dona de um artefacto (classes, casos de uso, esboços, serviços e restrições, atividades), com o relatório e os slides repartidos.
Alternativas	Dividir por grupo funcional (cada pessoa faz todos os diagramas de uma parte da aplicação).
Porquê	Cada diagrama UML precisa de coerência interna e de um autor; dividir por funcionalidade dava diagramas desenhados a várias mãos, com estilos e nomes diferentes. O diagrama de classes, o mais pesado, ficou sozinho; as restrições passaram para quem tinha menos carga.
Base	Plano da Etapa 1 (`Projeto/Plano-Etapa1.md`).

D4 Glossário de entidades como primeiro entregável

Decisão	Antes de qualquer diagrama, um glossário com os nomes das classes e dos atributos.
Alternativas	Cada pessoa usar os seus nomes e harmonizar no fim.
Porquê	Todos os artefactos usam os mesmos dados. Um glossário único evita incoerências entre o modelo, os fluxos, os esboços e as restrições, e desbloqueia os outros membros no primeiro dia.
Base	`etapa1/classes/glossario.md`.

D5 Revisão cruzada entre pares

Decisão	Cada artefacto é revisto por outro membro (Rodrigo↔Edson, Edson↔Miguel, Miguel↔Reis, Reis↔Ze, Ze↔Rodrigo).
Porquê	Deteta incoerências cedo e garante que pelo menos duas pessoas conhecem cada parte, o que ajuda se alguém faltar na apresentação.

D6 **Estimativa de esforço de três pontos**	
Decisão	Para cada tarefa: otimista (O), provável (M) e pessimista (P); esforço esperado E = (O + 4M + P) ÷ 6.
Alternativas	Uma só estimativa por tarefa; opinião de especialista sem método.
Porquê	Dá um valor esperado e uma medida da incerteza (desvio-padrão a partir de (P − O) ÷ 6). Resultado: cerca de 86 h esperadas, entre cerca de 79 h e 94 h, com 20% por pessoa.
Base	Aula T06; Anexo B do relatório (Tabela 20); secção 2.1 do plano.
Se perguntarem	"De onde vêm O e P?" — O valor provável é o do plano; o otimista e o pessimista são estimativas da equipa.

D7 **Pressupostos quantitativos assinalados como estimativas**	
Decisão	Assumir volumes e tempos de resposta (1 000 utilizadores, 50 em simultâneo, páginas até 2 s, sugestões até 300 ms, 99% de disponibilidade, cerca de 20 compras/dia) e escrever explicitamente que são estimativas.
Alternativas	Não indicar números; inventar números como se fossem requisitos.
Porquê	O enunciado não os dá, mas o âmbito do produto precisa de dados quantitativos para orientar a RNF-08 e a RNF-10. Assinalá-los como estimativas evita apresentá-los como requisitos do cliente.
Base	Aula T05 (âmbito do produto); Tabela 3 do relatório.

D8 **Cenários dos casos de uso como medida de tamanho**	
Decisão	Contar o cenário principal e os alternativos de cada caso de uso (66 no total).
Porquê	Os casos de uso têm níveis de abstração diferentes (uns juntam duas US); o número de cenários é uma medida mais justa e serve de base aos Use Case Points e à decomposição na Etapa 2.
Base	Aula T05; Tabela 7 do relatório.

2 Arquitetura e tecnologias

D9 **Arquitetura cliente-servidor em três camadas**	
Decisão	Frontend (no browser), backend com API e base de dados, independentes entre si.
Porquê	É obrigatório (RNF-01). Separar as camadas permite desenvolver e testar frontend e backend em paralelo, e escalar o backend sem mexer no frontend.
Base	Enunciado, secção 1; RNF-01.

D10 **Stack MEAN**	
Decisão	MongoDB, Express, Angular e Node.js.
Alternativas	Outras stacks (por exemplo, React ou Vue no frontend; Django ou Spring no backend; PostgreSQL na base de dados).
Porquê	É a stack sugerida no enunciado, o que evita a pesquisa de uma stack alternativa na Etapa 1. Usa JavaScript/TypeScript nas três camadas, e os dados circulam em JSON do browser à base de dados, o que simplifica a API.
Base	Enunciado, secção 1 e secção 6 ("caso a equipa opte por uma stack diferente...").
Se perguntarem	"Porque não uma base de dados relacional?" — Ver D11.

D11 **MongoDB como base de dados**	
Decisão	Base de dados de documentos.
Alternativas	Base de dados relacional (PostgreSQL, MySQL).
Porquê	Faz parte da stack sugerida. Dados como as faixas de um álbum ou as linhas de uma fatura ficam bem como listas dentro do documento do álbum ou da fatura. O plano gratuito do MongoDB Atlas (512 MB) chega para os dados previstos (menos de 100 MB, Tabela 3).
Cuidado	O modelo de dados UML é independente da base de dados; a forma como cada associação fica em MongoDB (documento embebido ou referência) decide-se na implementação.

D12 **Angular no frontend e Express/Node.js no backend**	
Decisão	Angular para a interface; Express sobre Node.js para a API.
Porquê	São os da stack sugerida. O Angular traz estrutura (componentes, formulários com validação, routing), útil para os muitos formulários com regras (US1, US22) e para o design responsivo (RNF-04). O Express é minimalista e adequado a uma API REST (RNF-09).

D13 **API REST**	
Decisão	O backend expõe uma API REST, com recursos, verbos HTTP e códigos de estado corretos.
Porquê	A RNF-09 exige uma API REST com boas práticas; a descrição em OpenAPI faz parte da Etapa 2.
Base	RNF-09; enunciado, secção 6 (Etapa 2).

D14 **Backend sem estado de sessão**	
Decisão	A sessão é identificada por um token; o servidor não guarda estado da sessão.
Porquê	Permite acrescentar nós ao backend se a carga crescer (RNF-10) e tolerar a falha de um nó (RNF-11). A RNF-12 pede uma sessão autenticada antes dos pedidos com dados do utilizador.
Base	RNF-10, RNF-11, RNF-12; Tabela 3.

D15 **Palavra-passe guardada cifrada e mostrada mascarada**	
Decisão	Guardar só uma versão cifrada (`passwordHash`); no perfil a palavra-passe aparece como "••••••••".
Alternativas	Guardar a palavra-passe em claro para a poder mostrar (US3 a).
Porquê	A RNF-12 recomenda cifrar as palavras-passe; depois de cifrada não é possível mostrá-la. O conflito com a US3 foi colocado ao professor (Q3).

D16 **Mensagem de erro genérica no início de sessão**	
Decisão	Se as credenciais estiverem erradas, a mensagem não diz se o username existe.
Porquê	Exigido pela US2 b); impede que alguém descubra que contas existem.

D17 **Custo gratuito e código aberto como critérios de escolha**	
Decisão	Preferir serviços com plano gratuito e bibliotecas de código aberto.
Porquê	A RNF-15 pede custo idealmente gratuito e a RNF-05 código fonte aberto. Estes dois critérios aparecem em todas as comparações de serviços.

3 Serviços externos

Todas as escolhas foram comparadas com pelo menos um concorrente, como pede o enunciado, com preços e condições consultados a 27 e 29 de outubro de 2026.

D18 Mapas: Leaflet + OpenStreetMap

Alternativas	Google Maps (10 000 carregamentos grátis/mês, exige conta com faturação); Mapbox (cerca de 50 000 grátis/mês, licença proprietária).
Porquê	Gratuito, sem cartão de crédito, de código aberto (RNF-05) e sugerido no enunciado (nota 9). As user stories só precisam de mostrar marcadores (US3, US29).
Se perguntarem	"O Google Maps não é melhor?" — Tem mais funcionalidades, mas não as usamos, exige faturação e restringe guardar dados.

D19 Geocodificação: Nominatim

Alternativas	Google Geocoding (restringe guardar resultados); Mapbox Geocoding (guardar resultados é pago).
Porquê	O modelo guarda as coordenadas na Morada; o Nominatim permite-o (ODbL, com atribuição). Só se geocodifica ao criar a conta ou mudar a morada, muito abaixo do limite de 1 pedido por segundo.
Cuidados	Um pedido por morada, com o resultado guardado; sem preenchimento automático; só se envia rua, código postal e localidade, sem o nome; atribuição ao OpenStreetMap.

D20 Pagamentos: Stripe e PayPal (os dois)

Alternativas	Um só serviço.
Porquê	A US28 b) e g) pede que o utilizador escolha o serviço e possa trocar se o pagamento falhar, por isso são precisos dois. O Stripe tem taxas mais baixas para cartões europeus (1,5% + 0,25 €) e aceita MB WAY; o PayPal serve quem paga com a conta PayPal.

D21 Página de pagamento alojada e confirmação por webhook

Decisão	O utilizador paga na página do serviço; o serviço avisa o backend do resultado.
Porquê	A aplicação nunca trata dados de cartão (RNF-12). A US28 c) diz que a aplicação deve aguardar o resultado do serviço antes de emitir a fatura.

D22 Imagens de capa no Cloudinary

Alternativas	Guardar na base de dados (documento ou GridFS); usar só o Cover Art Archive.
Porquê	O gestor tem de poder carregar qualquer capa (US22 f). O MongoDB Atlas gratuito tem só 512 MB para todos os dados, e as capas ocupam cerca de 0,6 GB; o Cloudinary tem 25 créditos grátis por mês, gera miniaturas para telemóvel (RNF-04) e entrega por CDN (RNF-08).

D23 Cover Art Archive só para sugerir a capa

Porquê	Indexado pelo MBID, que o modelo já guarda; poupa trabalho ao gestor. Não basta sozinho, porque nem todos os álbuns têm capa lá, e a documentação não define a licença das imagens (direitos dos titulares). O gestor confirma a sugestão antes de guardar.

D24 **Faixas e canções importadas da API do MusicBrainz**	
Alternativas	Introdução manual pelo gestor (exigiria user stories extra).
Porquê	Nenhuma US obrigatória regista canções, mas a US9 b) mostra as faixas; a nota 12 sugere importação em bloco. O MusicBrainz usa o MBID, os dados necessários são CC0 e um álbum inteiro vem num pedido (limite de cerca de 1 pedido/s, User-Agent com contacto).
Pontos em aberto	As faixas pertencem a um *release* (edição), não ao *release group* (o nosso Album); nem todas as gravações têm ISRC, que no modelo identifica a canção (RI-34).

D25 **Pagamentos só em modo de teste**	
Porquê	O projeto é académico: não há transações reais. Stripe e PayPal têm modo de testes completo (chaves e cartões de teste, sandbox).

4 Casos de uso

D26 Dois diagramas, um por ator humano

Alternativas	Um só diagrama com os 21 casos de uso.
Porquê	Num só diagrama as linhas cruzavam-se por todo o lado e ficava ilegível. Os casos de uso comuns (UC-02 e UC-09) aparecem nos dois.

D27 Critério de agrupamento das user stories

Decisão	As US que atuam sobre os mesmos dados e pelo mesmo ator formam um caso de uso: 29 US → 21 casos de uso.
Porquê	Evita casos de uso minúsculos (ver, ordenar, remover a mesma coleção) e mantém o diagrama legível. Exemplo: UC-10 Gerir coleção = US10 + US11.

D28 UC-08 «extend» UC-07 e UC-20 «extend» UC-09

Porquê	«extend» é comportamento opcional dentro de outro caso de uso: o favorito gere-se a partir da página do artista (US7 a); o gestor altera o stock a partir da página do álbum (US25 a). Não se usou «include» porque não há comportamento obrigatório partilhado que valha a pena separar.

D29 Gestor ligado ao UC-09

Porquê	A US25 a) faz-se "a partir da página de um álbum", por isso o gestor também visualiza essa página.

D30 Serviços externos como atores «sistema externo»

Porquê	O enunciado pede que os serviços externos apareçam no diagrama, ligados aos casos de uso que os usam. Usou-se uma caixa «sistema externo» para os distinguir dos atores humanos.

D31 Matriz US ↔ UC

Porquê	Prova que as 29 user stories estão todas cobertas, uma verificação que o professor pode pedir.

5 Modelo de dados

19 classes em 5 grupos. A classe central é a **VersaoAlbum**: o álbum num suporte concreto, com EAN-13, que se compra, coleciona e tem stock.

D32 Diagrama de topo sem atributos, com fundos por grupo, e um diagrama por grupo

Porquê	É o que o enunciado pede quando o diagrama não cabe numa página A4 (secção 6 e nota 18).

D33 Solista e Grupo como subclasses de Artista

Alternativas	Um atributo "tipo" em Artista e uma restrição escrita.
Porquê	A associação `membros` liga Grupo a Solista, por isso o próprio modelo impede um grupo de ter grupos como membros, ou a si próprio (secção 2 do enunciado; RI-31).

D34 Membros de grupos com multiplicidade 0..*

Porquê	O registo de membros é uma funcionalidade extra (nota 11): o modelo suporta-o, mas não o torna obrigatório.

D35 Faixa como classe própria

Porquê	A mesma canção está em vários álbuns com números diferentes ("Last Resort" no "Infest" e no "Now 4"). O número pertence à ligação álbum–canção; pô-lo na canção é impossível, e duplicar a canção violava a unicidade do ISRC (RI-34).

D36 Album com 0..1 artista

Porquê	As compilações não têm artista; a aplicação mostra "Vários artistas" (nota 10; RI-32).

D37 Album com 0..* faixas

Porquê	O registo do álbum (US22) não inclui as canções; as faixas são importadas depois (D24).

D38 VersaoAlbum separada de Album

Porquê	O mesmo álbum existe em CD, vinil, cassete e em várias edições, cada uma com o seu EAN-13 (secção 2). É a versão que se compra e coleciona.

D39 DisponibilidadeLoja separada e opcional (0..1)

Porquê	Distingue os três casos da US9: nunca esteve na loja (não existe), disponível (unidades > 0) e esgotado (0 unidades e data da última venda). Evita atributos vazios em versões que nunca estiveram à venda.

D40 dataUltimaVenda é a "data de fim de disponibilidade"

Porquê	A RI-17 diz que a data de fim de disponibilidade é igual à data da última venda; guardar as duas seria redundante.

D41 Preço guardado na LinhaFatura

Porquê	O preço na loja pode mudar; a fatura tem de manter o preço pago.

D42 Fatura com cópia dos dados de faturação

Decisão	A Fatura guarda `nifFaturacao`, `nomeFaturacao` e `moradaFaturacao`.
Porquê	A US28 d) exige os dados do utilizador na fatura, e estes não podem mudar se o perfil for alterado (RI-48).

D43 Utilizador 0..1 do lado da Fatura

Porquê	A RNF-16 permite ao utilizador eliminar os seus dados; as faturas têm de ser conservadas, por isso sobrevivem à conta com a cópia dos dados.

D44 A coleção guarda versões, as listas guardam álbuns

Porquê	A US10 adiciona "uma versão" à coleção e o enunciado diz que se adiciona "cada um dos suportes"; a US15 diz que as listas são "independentemente das versões" e a US17 adiciona "um álbum".

D45 ItemColecao ligado à LinhaFatura de origem (0..1)

Porquê	Distingue os itens comprados na loja dos adicionados à mão, útil nas estatísticas (US29). A RI-51 garante a coerência (mesma versão, mesmo utilizador, data da fatura).

D46 Conta como superclasse de Utilizador e Gestor

Porquê	Ambos iniciam sessão com username e palavra-passe; só o utilizador tem dados pessoais, coleção, listas, carrinho e pedidos. A conta do gestor já vem criada.

D47 Morada como tipo de dados com coordenadas opcionais

Porquê	A morada não tem identidade própria (é um valor do utilizador); as coordenadas são opcionais porque a US3 só mostra o mapa "se for possível" geocodificar.

D48 Notificação ligada ao pedido, com o atributo lida

Porquê	O destinatário e a resposta já estão no pedido, por isso não se repetem. O atributo `lida` permite o contador de notificações por ler no dashboard (US14 a; RI-50).

D49 PedidoNovaVersao com dataResposta e ligação à versão criada

Porquê	Regista quando o gestor respondeu (RI-49) e, se aceitou, qual a versão criada (US24 c).

6 Restrições de integridade

D50 Restrições nas notas das classes

Porquê	O enunciado pede as restrições no diagrama de classes; as notas ligadas a cada classe mostram onde cada uma se aplica. As que envolvem várias classes (RI-06, RI-10, RI-11) estão descritas no texto.

D51 21 restrições novas, todas com origem

Decisão	RI-31 a RI-51, cada uma com a frase do enunciado, a US ou a RNF de onde vem.
Porquê	O enunciado prevê restrições além das 30 dadas, com identificadores a partir de RI-31. Indicar a origem prova que nenhuma foi inventada.
Se perguntarem	"Dê um exemplo." — RI-41: cada versão só uma vez na coleção (US10 c); RI-46: só vai para o carrinho o que tem stock (US26 a).

D52 Anexo A: cada restrição associada a uma classe

Porquê	Verificação de que as 51 restrições têm um sítio no modelo, o que facilita a implementação das validações em PTIR (RNF-02).

7 Fluxos de trabalho

D53 Fluxos escolhidos

Decisão	Comprar (com pagamento como subatividade), pedido de nova versão, criar conta e iniciar sessão, gerir o catálogo.
Porquê	São os que envolvem vários atores ou várias user stories, que o enunciado destaca como os mais difíceis.

D54 Pagamento decomposto numa subatividade

Porquê	O enunciado pede que fluxos que não cabem numa página sejam decompostos; o pagamento é o passo mais complexo da compra.

D55 Notação UML de diagrama de atividades

Decisão	Raias por ator; decisões e merges em losango vazio com guardas `[ ]`; fork/join; nós inicial e final.
Porquê	O enunciado diz que os diagramas seguem a notação UML. As guardas tornam as condições explícitas; o fork/join mostra o que pode correr em paralelo.

D56 Reserva das unidades antes de pagar

Porquê	Dois utilizadores podem estar a comprar a última unidade ao mesmo tempo. A reserva garante que só um a consegue; se uma versão esgotou entretanto, sai do carrinho e o utilizador é avisado.
Em aberto	Como se guarda a reserva e quando expira ainda não está no modelo de dados: decide-se na Etapa 2.

D57 Guarda de tempo limite no pagamento

Porquê	O serviço externo pode não responder; sem tempo limite, as unidades ficavam reservadas para sempre. Ao esgotar o tempo, a reserva é libertada.

D58 Fatura emitida antes do fork

Decisão	Emitir a fatura; depois, em paralelo, adicionar à coleção e confirmar a saída do stock.
Porquê	Os itens da coleção ficam ligados às linhas da fatura (RI-51), por isso a fatura tem de existir primeiro; a coleção e o stock são independentes entre si.

D59 "Cancelar o pagamento" na raia do Utilizador

Porquê	É uma escolha do utilizador (US28 g), não da aplicação.

D60 Cifrar a palavra-passe e geocodificar em paralelo

Porquê	São ações independentes; o fork/join mostra-o e evita que a criação da conta espere sem necessidade pelo serviço externo.

8 Esboços da interface

D61 Esboços de baixa fidelidade

Alternativas	Protótipos de alta fidelidade, com o aspeto final.
Porquê	O objetivo é perceber a complexidade das user stories antes de implementar, não definir o aspeto gráfico; os de baixa fidelidade são mais rápidos e focam a discussão na estrutura.

D62 Ecrãs escolhidos: os mais complexos

Porquê	O enunciado diz que o foco deve estar nas funcionalidades mais complexas. A página de álbum junta 5 user stories; o carrinho e pagamento envolvem um serviço externo; o perfil e as estatísticas usam mapas.

D63 Notas numeradas ligadas às user stories

Porquê	Cada elemento do ecrã fica justificado por uma US ou restrição, o que dá rastreabilidade e facilita a apresentação.

D64 Duas versões para telemóvel

Porquê	A RNF-04 pede design responsivo; mostrou-se a adaptação dos dois ecrãs mais carregados (tabelas passam a cartões, botão Pagar fixo).

D65 Campo da morada sem sugestões automáticas

Porquê	A política de utilização do Nominatim proíbe o preenchimento automático.

D66 Coluna do gestor e botões por versão na página de álbum

Porquê	Coleção e carrinho atuam sobre a versão (D44); "Adicionar a uma lista" atua sobre o álbum. O botão do carrinho só fica ativo com stock (RI-46). A coluna do gestor implementa o UC-20 «extend» UC-09.

9 Ferramentas de trabalho

Ferramentas usadas para **produzir** os artefactos da Etapa 1 (não fazem parte da aplicação).

D67 Git e GitHub, com repositório privado

Porquê	Controlo de versões para trabalhar em equipa (antecipa a RNF-06), histórico de alterações e partilha de todos os artefactos. Privado para o trabalho não ficar visível a outros grupos.

D68 Uma pasta por artefacto e por responsável

Porquê	Evita que duas pessoas editem o mesmo ficheiro e deixa claro quem é dono de quê.

D69 Diagramas como texto (fonte no repositório)

Alternativas	Desenhar à mão no draw.io (o plano inicial).
Porquê	A fonte em texto fica versionada no Git, as alterações ao modelo regeneram todos os diagramas de forma coerente, e qualquer membro consegue repetir a geração.

D70 Graphviz para os diagramas de classes; gerador próprio para o de topo

Alternativas	Mermaid (o primeiro formato usado).
Porquê	O Mermaid cortava multiplicidades e não fazia fundos coloridos por grupo; o Graphviz faz os fundos (clusters) e controla as pontas das associações. O diagrama de topo, com muitas associações a convergir, foi desenhado com posições fixas para não haver sobreposições.

D71 Mermaid para os diagramas de atividades

Alternativas	PlantUML.
Porquê	O Mermaid tem as formas UML necessárias (decisão, fork/join, nós inicial e final) e raias; o PlantUML obrigaria a instalar uma ferramenta extra. As formas dos nós inicial e final são acertadas no passo de exportação.

D72 Relatório escrito em Markdown e convertido para Word e PDF

Porquê	Uma só fonte de texto, versionada no Git e fácil de rever; o Word e o PDF saem sempre iguais e com a numeração verificada por script.

D73 Imagens em SVG e PNG

Porquê	O SVG não perde qualidade no Word; o PNG serve para slides e pré-visualização.

D74 Apresentação com notas do orador

Porquê	Cada slide indica quem apresenta e o que dizer, para todos poderem estudar as partes uns dos outros.

10 Suposições

Pontos em que o enunciado é ambíguo ou omisso. Foram colocados ao professor (Q1 a Q4) e, até haver resposta, a equipa segue estas suposições.

ID	Problema	Suposição e porquê
S1 (Q1)	A US28 f) está cortada a meio.	Regista-se a data da última venda (RI-17), que é o que a RI-17 já pede quando a versão esgota.
S2 (Q2)	"Recusado" (US13, US14) ou "rejeitado" (US24)?	"Recusado", porque aparece em duas US e na RI-44.
S3 (Q3)	A US3 mostra a palavra-passe; a RNF-12 recomenda cifrá-la.	Cifrada e mostrada mascarada: a segurança prevalece.
S4 (Q4)	Nenhuma US obrigatória regista canções.	Importação em bloco com o MusicBrainz, como sugere a nota 12.
S5	O enunciado não fala de quantidades no carrinho nem na fatura.	Cada linha é uma unidade de uma versão; não se acrescentou um requisito que não existe.
S6	Não há volumes nem tempos de resposta.	Pressupostos da Tabela 3, assinalados como estimativas da equipa (D7).

11 Resumo numa página

Área	Decisões principais (e a razão em poucas palavras)
Processo	Scrum (cliente participativo, equipa de 5, requisitos mutáveis, 3 sprints); gestor como PO parcial; estimativa de três pontos (86 h, 20% cada).
Arquitetura	3 camadas (RNF-01); MEAN (sugerida, JSON de ponta a ponta); API REST (RNF-09); backend sem estado (RNF-10); palavra-passe cifrada (RNF-12).
Serviços	Leaflet+OSM (grátis, aberto); Nominatim (deixa guardar coordenadas); Stripe+PayPal (US28 obriga a escolher); Cloudinary + Cover Art Archive (capas); MusicBrainz (faixas, CC0).
Casos de uso	2 diagramas (legibilidade); agrupar por dados e ator; 2 «extend»; serviços como sistemas externos; 66 cenários para estimar.
Modelo	VersaoAlbum como centro; Solista/Grupo; Faixa; disponibilidade opcional; cópia dos dados na fatura; coleção de versões e listas de álbuns.
Restrições	51 (30 + 21), todas com origem e associadas a uma classe.
Fluxos	UML com raias, guardas e fork/join; reserva das unidades e tempo limite no pagamento; fatura antes do fork.
Esboços	Baixa fidelidade, ecrãs mais complexos, notas ligadas às US, versões para telemóvel.
Ferramentas	Git/GitHub privado; diagramas como texto (Graphviz, Mermaid); relatório em Markdown → Word/PDF.